Levenberg-Marquardt Calibration

Gradient-based parameter estimation with uncertainty in OpenHydroQual

Contents

1 Overview **1**

2 What is computed **2**
2.1 The objective . . . 2
2.2 The residual vector . . . 2
2.3 The step . . . 3
2.4 Convergence . . . 3
2.5 Parameter uncertainty . . . 3
2.6 Error standard deviations . . . 4
2.7 Autocorrelated errors (AR1) . . . 4

3 Requirements and limitations **5**

4 Settings **5**

5 Using LM in the GUI **6**

6 Using LM from the console **6**
6.1 Script syntax . . . 6
6.2 Headless runner: `OHQ-LM` . . . 7
6.3 Checking the likelihood decomposes . . . 7

7 Output files **7**
7.1 `LM_output.txt` . . . 7
7.2 `LM_covariance.txt` . . . 8

8 Worked example **8**

9 Practical notes **9**
9.1 The finite-difference step . . . 9
9.2 Parameters that sit on a bound . . . 9
9.3 Parameters the data cannot see . . . 9
9.4 Comparing objective values between runs . . . 9

1 Overview

Levenberg-Marquardt (LM) estimates model parameters by walking downhill from a starting guess to the best-fitting values, using the slope of the fit error to choose each step. It minimises exactly the same negative log-likelihood that **Inverse Run** (genetic algorithm) and **MCMC parameter estimation** minimise, so the three methods are directly comparable and can be used in sequence on the same model without changing anything about the model itself.

What distinguishes LM from its two siblings:

- **It is cheap.** One iteration costs roughly one model run per calibrated parameter. A genetic algorithm costs population × generations, typically 1600 runs at the default settings.
- **It reports uncertainty.** At the solution it produces a standard error and a 95 % interval for every parameter, plus the correlation matrix between them. The GA reports a point estimate and nothing else; MCMC reports uncertainty but needs thousands of runs to do it.
- **It is local.** It finds the optimum nearest its starting point. On a model with several distinct optima it will not search for the global one.

That last point defines the intended workflow. LM starts from each parameter's *current value*, and **Inverse Run** writes its estimates back into the model's parameters when it finishes. Running LM immediately after it therefore continues from the GA's answer with no configuration at all:

> **Inverse Run** finds the basin. **Levenberg-Marquardt** converges inside it and quantifies the uncertainty.

Used alone from a good starting guess, LM is often all that is needed. Used alone from a poor one on a multi-modal problem, it will return a confident answer about the wrong optimum, which is the one failure mode to keep in mind.

2 What is computed

2.1 The objective

Every `Observation` with observed data contributes a Gaussian negative log-likelihood. For an observation compared at n points,

$$-\log L \;=\; \frac{n}{\tau}\left(\frac{\mathrm{MSE}}{2\sigma^2}+\log\sigma\right), \qquad \mathrm{MSE} \;=\; \frac{1}{n}\sum_{i=1}^{n}(o_i-m_i)^2\,, \tag{1}$$

where o_i and m_i are the observed and modeled values, σ is the observation's `error_standard_deviation`, and τ is its effective-information scale. For a `log-normal` error structure the residuals are taken in log space. The total objective is the sum over all observations. This is the same quantity `System::CalcMisfit()` returns to the GA and to MCMC.

2.2 The residual vector

LM does not minimise that scalar directly. It decomposes it into a vector of weighted residuals

$$r_i \;=\; \frac{o_i-m_i}{\sigma\sqrt{\tau}}, \qquad \text{so that} \qquad -\log L \;=\; \tfrac{1}{2}\|\mathbf{r}\|^2 + n\,\log\sigma, \tag{2}$$

with an extra factor $\sqrt{w_i}$ per point under the `Weighted Least Squared` kernel. Every scaling the misfit applies is folded into $\mathbf{r}$, so the residual vector LM differentiates and the misfit the GA minimises are two views of one quantity, computed by one piece of code.

2.3 The step

With $J_{ij} = \partial r_i / \partial \theta_j$ estimated by finite differences, each iteration solves the damped normal equations

$$\left(J^\top J + \lambda \,\mathrm{diag}(J^\top J)\right)\, \boldsymbol{\delta} \;=\; -J^\top \mathbf{r}, \tag{3}$$

and tries the point $\boldsymbol{\theta} + \boldsymbol{\delta}$. Small λ gives the full Gauss-Newton step, which converges quickly near the solution; large λ gives a short step down the steepest-descent direction, which is safer far from it. The damping adapts automatically: it is reduced after a step that improves the fit and raised after one that does not. Scaling by $\mathrm{diag}(J^\top J)$ instead of by the identity makes the damping insensitive to how the parameters are scaled, which matters when conductivities and porosities are calibrated together.

Parameters are searched in the same space the GA uses: $\log_{10}$ for a parameter whose `prior_distribution` is `log-normal`, linear otherwise, bounded by its `low` and `high`. Steps that would leave the box are projected back onto it.

2.4 Convergence

The run stops at the first of these:

- the fit improves by less than `tol_objective` (relative);
- the step is shorter than `tol_step` (relative);
- the largest gradient component is below `tol_gradient`;
- no step improves the fit in `max_rejections` attempts at heavier damping, which means a minimum has been reached to within the resolution the finite-difference gradient can see;
- `max_iterations` is reached.

The stop reason is written in plain words at the end of `LM_output.txt`. Read it: "reached the iteration limit" means the run was cut short and should be continued, while the others mean it converged.

2.5 Parameter uncertainty

At the solution $J^\top J$ is the Gauss-Newton approximation of the likelihood's Hessian, so its inverse is the parameter covariance. Because the residuals already carry $1/\sigma$, this is a covariance and not merely a curvature. LM recomputes the Jacobian at the converged point (rather than reusing the last iteration's, which was taken one step away) and reports:

- the standard error of each parameter, in the space it was searched in, so a `log-normal` parameter's error refers to $\log_{10}$ of its value;
- a 95 % interval, converted back to model units;
- the correlation matrix between parameters.

Read the correlation matrix. A pair of parameters correlated near $+1$ or -1 cannot be told apart by the available data: the fit can trade one against the other with no penalty. The individual standard errors will look reasonable while the pair is jointly unidentified. Fix one of them, or add an observation that constrains one alone.

2.6 Error standard deviations

When a parameter drives an observation's `error_standard_deviation`, the objective is no longer a pure sum of squares and σ would have to be searched for like any other parameter. LM avoids this: σ has a closed form at any point,

$$\hat{\sigma}^2 \;=\; \mathrm{MSE}, \tag{4}$$

so LM sets it to that value at every iteration and drops it from the search. This is exact, costs nothing, removes a parameter from the problem, and is the same value the GA converges to. It is on by default (`profile_sigma`); the estimate reported at the end is the profiled one.

2.7 Autocorrelated errors (AR1)

Residuals of a hydrological model are serially correlated: an error in the timing or volume of an event, or a baseflow bias, persists over many observations. Treating them as independent counts n correlated residuals as n independent ones, which makes parameter uncertainty far too small and lets long, smooth stretches of record dominate the fit. An observation can instead be given a first-order autoregressive error model by setting its `error_autocorrelation` property to `AR1`; the default, `none`, is the independent model above, unchanged.

Model. With e_i the residual at time t_i (in the space set by `error_structure`, i.e. the log residual for log-normal errors), the errors follow a continuous-time AR1 (Ornstein–Uhlenbeck) process with correlation time τ (`error_correlation_time`, days):

$$e_i = \phi_i\, e_{i-1} + \eta_i, \qquad \phi_i = e^{-(t_i - t_{i-1})/\tau}, \qquad \eta_i \sim N\Big(0,\ \sigma^2(1-\phi_i^2)\Big), \qquad e_1 \sim N(0,\sigma^2). \tag{5}$$

σ (`error_standard_deviation`) is the marginal, stationary standard deviation, so it keeps its meaning. Because ϕ_i depends on the spacing, irregularly sampled data (hourly during events, daily otherwise) and gaps between observation windows are handled consistently: across a long gap $\phi_i \to 0$ and the series restarts. $\tau = 0$ is the independent model.

Likelihood. The exact negative log-likelihood is

$$-\ln L \;=\; n\ln\sigma \;+\; \sum_{i\ge 2} \ln\sqrt{1-\phi_i^2} \;+\; \frac{1}{2\sigma^2}\left[e_1^2 + \sum_{i\ge 2}\frac{(e_i - \phi_i e_{i-1})^2}{1-\phi_i^2}\right] + \text{const}, \tag{6}$$

i.e. the independent likelihood of the *whitened* residuals $u_1 = e_1$, $u_i = (e_i - \phi_i e_{i-1})/\sqrt{1-\phi_i^2}$, plus a log-determinant term that depends only on τ and the observation times. The residual vector LM differentiates is $r_i = u_i/\sigma$, and the decomposition of Section 6.3 becomes $-\ln L = \frac{1}{2}\|r\|^2 + n\ln\sigma + \sum \ln\sqrt{1-\phi_i^2}$ (`-verify-residuals` reports the log-determinant term). ϕ_i is capped at 0.9999 and repeated time stamps are treated as uncorrelated. The fit measures (MSE, R^2, NSE) are still reported for the raw residuals.

Fixed or estimated. σ and τ are ordinary observation properties, one each per observation: they can be given values or driven by a parameter with `setasparameter` (quantity `error_correlation_time`). The GA and MCMC search or sample τ like any parameter. LM profiles it, as it does σ: a parameter that drives only correlation times is removed from the Gauss–Newton vector and set, after every accepted step, by a golden-section search over its range on the stored raw residuals (no model solve), with every profiled σ re-profiled at each trial value. Holding σ fixed while τ moves would make every step in τ look worse than it is, because the optimal σ (the RMS of the whitened residuals) changes with τ. On synthetic AR1 data (1780 irregularly spaced points, $\sigma = 0.3$, $\tau = 0.7$ d) LM returns the maximum-likelihood values of a direct fit to all digits ($\hat{\sigma} = 0.3178$, $\hat{\tau} = 0.808$ d).

Not combined with `likelihood_scale`. The likelihood scale corrects for the same autocorrelation by dividing the log-likelihood by an integral correlation time; with AR1 errors leave it at 1. AR1 is implemented for the `Least Squared` comparison only.

3 Requirements and limitations

LM refuses to start, with an explanatory message, unless all of the following hold. Nothing is run and no output is overwritten when it refuses.

- **The model defines observations with observed data.** LM needs residuals. It cannot score a design objective-function set, which is what **Optimize** is for; LM's counterpart is **Inverse Run**, the genetic algorithm scoring the same likelihood.
- **Every observation uses a comparison method with a sum-of-squares form.** `Least Squared`, `Weighted Least Squared` and `EMC` all qualify. `Similarity` does not: it is an autocorrelation and Kolmogorov-Smirnov distance with no Gauss-Newton form. Use **Inverse Run** for such a model.
- **The model solves at the starting parameter values.** LM has no population to fall back on. If the starting point fails, run **Inverse Run** first to reach a feasible region.
- **There are at least as many residuals as free parameters.** Otherwise the problem is underdetermined and the covariance is singular by construction.
- **The number of residuals does not change when a parameter is perturbed.** If it does, the comparison points themselves are moving with the parameters and a Jacobian is not defined. LM detects this and stops rather than differentiating nonsense.

4 Settings

The **LM** settings object holds the configuration. Every entry has a usable default, so a model that has never been configured for LM will run.

Property	Default	Meaning
`max_iterations`	50	Most Gauss-Newton steps attempted. Each costs about one model run per parameter. A well-behaved problem stops well inside 50 on its own.
`lambda0`	0.01	Initial damping. Small takes the full Gauss-Newton step, good when already close; large takes short downhill steps, safer from a poor start. It adapts after the first step, so this mostly affects the opening moves.
`fd_step`	0.001	How far each parameter is nudged to measure its effect, as a fraction of that parameter's range. See §9.1.
`central_differences`	No	Nudge each parameter both up and down. Twice the model runs per iteration, for a more accurate gradient.
`profile_sigma`	Yes	Set a calibrated `error_standard_deviation` to its analytic best value each iteration instead of searching for it.

Property	Default	Meaning
`lm_numthreads`	8	Perturbed models solved at once while measuring the gradient. Nothing is gained from more threads than parameters, or than the machine has cores.
`tol_objective`	1e-8	Stop when an iteration improves the fit by less than this fraction.
`max_rejections`	10	How many times a failed step is retried with heavier damping before the run concludes it has reached a minimum. Each retry is a full model run.
`lm_outputfile`	`LM_output.txt`	Iteration log.
`covariancefile`	`LM_covariance.txt`	Standard errors, intervals and the correlation matrix.

Further properties are settable from a script or the command line but are not exposed in the dialog, because the defaults are almost always right: `lambda_up` and `lambda_down` (both 3, the factors by which damping is raised after a rejected step and lowered after an accepted one), `lambda_max` (1e12), `tol_step` and `tol_gradient` (both 1e-8), and `write_covariance` (Yes).

5 Using LM in the GUI

1. Define the parameters to calibrate, with a `low`, a `high` and a `prior_distribution`, and assign each to the model quantity it drives.
2. Define at least one `Observation` with observed data.
3. Set the parameters' `value` fields to your starting guess. This is where LM begins. If you have just run **Inverse Run**, they already hold the GA's estimates and there is nothing to do.
4. Adjust the **LM** settings object if needed.
5. Press the **Levenberg-Marquardt calibration** button in the general toolbar.

The progress window plots the negative log-likelihood against iteration number. The secondary progress bar tracks the perturbed model runs within the current Jacobian. When the run finishes, the calibrated values are written back into the model's parameters and the results are transferred in, so the plots show the calibrated model. The log reports the iteration count, the number of model runs consumed, and where the covariance file was written.

If LM refuses to start, a dialog explains which of the conditions in §3 was not met. Nothing has been run at that point.

6 Using LM from the console

6.1 Script syntax

LM settings are set like any other settings object, and the calibration is run with the `lmoptimize` command:

```
setvalue; object=LM, quantity=max_iterations, value=30
setvalue; object=LM, quantity=central_differences, value=Yes
setvalue; object=LM, quantity=numthreads, value=4
lmoptimize
```

Because LM starts from the parameters' current values and `optimize` writes its result back into them, the hybrid is simply the two commands in order:

```
optimize
lmoptimize
```

6.2 Headless runner: `OHQ-LM`

```
cd terminal/OHQ-LM && mkdir -p build && cd build
qmake6 ../OHQ-LM.pro && make -j$(nproc)

./OHQ-LM model.ohq output_folder/
```

Options, each overriding the model's **LM** settings when given and leaving them alone when not:

Option	Effect
`--verify-residuals`	Check the likelihood decomposition and exit. See §6.3.
`--iterations <n>`	Maximum iterations.
`--lambda0 <x>`	Initial damping.
`--fd-step <x>`	Finite-difference step, as a fraction of range.
`--central`	Central differences.
`--no-profile-sigma`	Keep a calibrated σ in the search.
`--threads <n>`	Parallel model runs per Jacobian.
`--kernel <lib.so>`	Run the forward model from a codegen-generated shared library instead of the interpreter.

6.3 Checking the likelihood decomposes

```
./OHQ-LM model.ohq output_folder/ --verify-residuals
```

This costs one forward solve and confirms that $\frac{1}{2}\|\mathbf{r}\|^2 + \sum_k n_k \log \sigma_k$ equals `CalcMisfit()` on this model, which is the identity the whole method rests on. It also reports, per observation, how many points were actually compared, the σ in force, and that observation's contribution to the objective.

It is worth running once on any new model. A comparison count well below the number of observed points means data lie outside the simulated time window and are not contributing at all.

7 Output files

7.1 `LM_output.txt`

A commented header records the settings, which parameters were optimised and in which space, and which error standard deviations were profiled out. Then one row per attempted step:

```
iter , neg_log_lik   , lambda     , status , k_decay, C0, sigma_obs
0    , -2.0052167e+01,  1.0000e-02, start  , 2, 3.5, 0.40935204
1    , -5.3282375e+01,  3.3333e-03, accept , 0.92588741, 1.7318153, 0.21336468
2    , -1.4003940e+02,  1.1111e-03, accept , 0.63504485, 1.9755143, 0.038934657
3    , -2.4437852e+02,  3.7037e-04, accept , 0.69347977, 2.0090912, 0.0050330171
```

`status` is `accept` when the step reduced the objective and `reject` when the damping had to be raised and the step retried. Parameter columns are in model units, not search units. The file ends with the stop reason, the final objective and the final estimates, with profiled parameters marked as such.

7.2 `LM_covariance.txt`

```
parameter    , space  , estimate   , std_error   , ci95_low  , ci95_high
k_decay      , log10  , 0.6941473  , 0.0008034   , 0.691635  , 0.6966687
C0           , linear , 2.01154    , 0.0025069   , 2.006626  , 2.016453

# correlation matrix
             , k_decay  , C0
k_decay      ,  1.0000  ,  0.6908
C0           ,  0.6908  ,  1.0000
```

The `space` column matters: for a `log-normal` parameter the standard error is that of $\log_{10}$ of the value, while the interval columns are converted back to model units.

This file is not written when the curvature at the solution is singular. That is itself a result: it means at least one parameter, or one combination of them, the data cannot resolve.

8 Worked example

A first-order decay in a batch reactor, $C(t) = C_0 e^{-kt}$, run to $t = 5$ with 51 observed points taken from the analytic solution. Two parameters are calibrated against their true values of $k = 0.7$ and $C_0 = 2.0$, plus the error standard deviation:

```
create parameter;type=Parameter,name=k_decay,low=0.05,high=5,value=2.0,
       prior_distribution=log-normal
setasparameter; object=decay_coefficient, quantity=base_value,
       parametername=k_decay

create parameter;type=Parameter,name=C0,low=0.5,high=6,value=3.5,
       prior_distribution=normal
setasparameter; object=Reactor (1), quantity=A:concentration, parametername=C0

create parameter;type=Parameter,name=sigma_obs,low=1e-4,high=10,value=1.0,
       prior_distribution=log-normal
setasparameter; object=exact_solution, quantity=error_standard_deviation,
       parametername=sigma_obs
```

Both parameters start far from the truth, at $k = 2.0$ and $C_0 = 3.5$. Running the genetic algorithm and LM separately on this model:

	Inverse Run (GA)	**Levenberg-Marquardt**
Settings	40 population, 40 generations	50 iterations allowed
Model runs	1643	**42**
$-\log L$	-242.168	$\mathbf{-244.879}$
k (truth 0.7)	0.6962	0.6941
C_0 (truth 2.0)	2.0170	2.0115
σ	0.004759	profiled
Standard errors	none	yes
Correlation	none	$\rho(k, C_0) = 0.69$

LM reached a slightly better optimum in roughly one fortieth of the model runs, and additionally reported that k and C_0 are 69 % correlated. That correlation is real and expected:

in $C_0 e^{-kt}$ a larger C_0 can be partly compensated by a faster k. It is the kind of thing a point estimate alone will never tell you.

The residual gap between the estimates and the exact values of 0.7 and 2.0 is the forward solver's discretisation error against the analytic solution, not a failure of the calibration: LM has found the best fit of the *discretised* model, which is the right answer to the question actually posed.

9 Practical notes

9.1 The finite-difference step

The forward model is solved with an adaptive time step, so the objective is only piecewise smooth. A perturbation smaller than the solver's own error measures numerical noise rather than a gradient. The default `fd_step` of 0.001 of each parameter's range clears this on typical settings.

If LM stalls at the first iteration with no step accepted, suspect the gradient before concluding the model is at an optimum. In order of what to try:

1. Increase `fd_step` to `0.01`.
2. Tighten the solver tolerance (`nr_tolerance`, `minimum_timestep`).
3. Turn on `central_differences`.

Conversely, if the calibration lands on a slightly different answer each time it is run from the same starting point, the gradient is being contaminated by solver noise and `central_differences` is the remedy.

9.2 Parameters that sit on a bound

A parameter driven against its `low` or `high` is held there by the projection, and its reported standard error is not meaningful: the likelihood is not locally quadratic at a boundary. Widen the range and re-run. If it returns to the same bound, the data are telling you the parameter wants to go outside the range you allowed.

9.3 Parameters the data cannot see

A parameter the residuals do not respond to produces a zero column in the Jacobian. LM holds such a parameter still rather than failing, and the singular curvature at the end suppresses the covariance file. Check that the parameter is actually assigned to a quantity that affects a modeled observation.

9.4 Comparing objective values between runs

The objective is a negative log-likelihood, so *lower is better* and the values are routinely negative once σ is profiled to a small value. Values are comparable between the GA, MCMC and LM on the same model, which is what makes the table in the worked example meaningful. They are not comparable across models with different observations or a different number of compared points.